

UNIVERSIDAD PRIVADA DEL NORTE

Facultad de Ingeniería

Carrera de Ingeniería de Sistemas Computacionales

Técnicas de Programación Orientada a Objetos (SIST1202A)

PRÁCTICA DE CAMPO 5

Diseño de Clases y Manejo de Archivos

Modelo UML, criterios de aceptación e implementación de los primeros 20 requerimientos

Proyecto: Sistema de Gestión y Valoración de Inventario Multialmacén

Dato	Detalle
Unidad	Unidad III: Programación visual y acceso a base de datos
Docente	CHRISTIAM JHON SANCHEZ ZAMORA
Equipo / grupo	Grupo 11
Repositorio	https://github.com/Brayanzamora/Sistema_gestion_valorizacion_inventario

Integrantes

N.º	Apellidos y nombres	Rol en el equipo	¿Aportó en el equipo?
1	ALARCÓN ZAMORA BRIAN RHOYERD	Developer	SI
2	ROJAS BRICEÑO WANDER	LT/Developer	SI
3	OCHOA ALMONACID ROY PAUL	Developer	SI
4	PRUDENCIO GUERRERO JHONATAN CESAR	Developer	SI
5	CORDOVA CESPEDES HILARY IVONNE		NO

Índice

1. Introducción.....	4
1.1 Contexto.....	4
1.2 Propósito.....	4
1.3 Alcance.....	4
1.4 Tecnologías.....	4
2. Arquitectura y decisiones de diseño.....	5
2.1 Arquitectura por capas.....	5
2.2 Decisiones de diseño.....	5
3. Diagrama de clases UML.....	7
3.1 Relaciones y multiplicidades.....	12
4. Responsabilidades y encapsulamiento	13
4.1 Responsabilidades por clase	13
4.2 Aplicación del encapsulamiento	14
5. Manejo de archivos	15
5.1 Clases de Java utilizadas	15
5.2 Formato de los archivos	15
5.3 Escritura y lectura (fragmentos del código)	16
5.4 Sincronización entre memoria y archivos.....	16
5.5 Evidencia de archivos generados	16
6. Matriz de trazabilidad de los 20 requerimientos.....	18
7. Evidencias de funcionamiento.....	23
7.1 Menú principal.....	23
7.2 REQ-01: Registrar nuevo producto	23
7.3 REQ-02: Listar productos activos	24
7.4 REQ-03: Actualizar precio unitario.....	24
7.5 REQ-04: Eliminar producto (baja lógica).....	24
7.6 REQ-05: Registrar nuevo almacén.....	24
7.7 REQ-06: Guardar productos en CSV	25
7.8 REQ-07: Cargar productos desde CSV	25
7.9 REQ-08: Registrar entrada de stock	25
7.10 REQ-09: Registrar salida de stock.....	25
7.11 REQ-10: Transferencia multialmacén	26
7.12 REQ-11: Bitácora de movimientos	26
7.13 REQ-12: Valoración total global	26

7.14 REQ-13: Valoración por almacén	26
7.15 REQ-14: Buscar producto por código exacto	27
7.16 REQ-15: Buscar por nombre parcial	27
7.17 REQ-16: Alerta de stock mínimo	28
7.18 REQ-17: Guardar almacenes en CSV	28
7.19 REQ-18: Cargar almacenes desde CSV.....	28
7.20 REQ-19: Filtrar movimientos por fecha	28
7.21 REQ-20: Exportar reporte de valoración.....	29
8. Registro de pruebas.....	30
9. Control de errores	32
9.1 Evidencias de errores controlados	32
10. Dificultades técnicas resueltas	34
11. Repositorio y trazabilidad de aportes	35
12. Compilación y ejecución	37
13. Cumplimiento de requerimientos mínimos, entregables y rúbrica.....	38
13.1 Requerimientos mínimos	38
13.2 Entregables.....	38
13.3 Relación con los criterios de la rúbrica	39
14. Conclusiones	40
Recomendaciones para la siguiente iteración	40
Anexo A. Diagrama de clases UML completo	41

1. Introducción

1.1 Contexto

En las prácticas anteriores se definieron los requerimientos y las historias de usuario del proyecto. En esta práctica, ese análisis se llevó a un diseño técnico: se definieron las clases del dominio, se implementó la persistencia en archivos y se verificó cada requerimiento contra sus criterios de aceptación.

1.2 Propósito

Diseñar el modelo de clases del **Sistema de Gestión y Valoración de Inventario Multialmacén** e implementar en Java sus primeros 20 requerimientos, con persistencia en archivos CSV y TXT, validaciones y criterios de aceptación verificables.

1.3 Alcance

Se entrega una primera versión funcional de consola, desarrollada en Java 17 sin frameworks externos. El sistema permite gestionar el catálogo de productos (registro, listado, actualización de precio y baja lógica), registrar almacenes con su stock y registrar entradas, salidas y transferencias entre almacenes. Cada movimiento se guarda en una bitácora.

Además, calcula la valoración del inventario total y por almacén, genera alertas de stock mínimo (menos de 10 unidades) y exporta un reporte de valoración. La información se almacena en la carpeta data/, en cuatro archivos: productos.csv, almacenes.csv, movimientos.txt y reporte_valoracion.txt.

1.4 Tecnologías

Elemento	Detalle
Lenguaje	Java 17 (maven.compiler.release = 17)
Build	Maven (pom.xml con maven-compiler-plugin, maven-jar-plugin y exec-maven-plugin). También compila con javac.
Persistencia	Archivos CSV y TXT con java.nio.file.Files, java.nio.file.Path, java.io.BufferedReader y java.io.BufferedWriter (UTF-8)
Interfaz	Menú de consola interactivo (Scanner) con 21 opciones y un modo --demo
Control de versiones	Git y GitHub

2. Arquitectura y decisiones de diseño

2.1 Arquitectura por capas

El código se organiza en cinco paquetes. La interfaz no accede a los archivos: cada operación pasa por la capa de servicio, que aplica las validaciones y delega la lectura y escritura a los repositorios. De este modo, un cambio en la forma de almacenamiento no afecta al menú.

Paquete	Clases	Responsabilidad
pe.upn.pc5.app	Main	Punto de entrada. Crea el servicio y decide entre el menú interactivo y el modo --demo.
pe.upn.pc5.ui	MenuConsola	Interacción con el usuario: lee y convierte las entradas, muestra resultados y captura excepciones.
pe.upn.pc5.service	InventarioService, DemoService	Reglas de negocio (validaciones, stock, valoraciones, alertas) y orquestación de la persistencia. DemoService genera las evidencias automáticas.
pe.upn.pc5.model	Producto, Almacen, Movimiento, TipoMovimiento	Entidades del dominio con encapsulamiento y validaciones propias.
pe.upn.pc5.persistence	ProductoRepository, AlmacenRepository, MovimientoRepository, ReporteRepository, RutasDatos	Lectura y escritura de archivos CSV y TXT. RutasDatos centraliza las rutas.

Las dependencias siguen un solo sentido: la interfaz depende del servicio y el servicio depende del modelo y de la persistencia. Las clases del modelo no dependen de ninguna otra capa.

2.2 Decisiones de diseño

Las principales decisiones de diseño y su justificación son las siguientes:

Decisión	Justificación
Baja lógica de productos (activo=false)	Evita que los movimientos registrados queden asociados a un producto inexistente. El producto se oculta de los listados y el historial se conserva.
Stock por almacén en un Map<String,Integer> dentro de Almacen	Cada almacén registra la cantidad de cada producto sin duplicar objetos Producto. El stock global se obtiene sumando los almacenes.
Stocks codificados en una columna (PROD01:20 PROD02:15)	Mantiene almacenes.csv con una fila por almacén, sin requerir un archivo adicional para los saldos.

Bitácora en modo append (movimientos.txt)	Cada movimiento se escribe al momento de registrarse, por lo que no se pierde si el programa se cierra de forma inesperada.
Identidad por código (equals/hashCode)	Dos productos o almacenes con el mismo identificador se consideran iguales, lo que evita duplicados.
Enum TipoMovimiento	Restringe los tipos de movimiento a ENTRADA, SALIDA y TRANSFERENCIA.
Códigos en mayúsculas	Los códigos 'prod01' y 'PROD01' identifican al mismo producto.
Rutas en una sola clase (RutasDatos)	La ubicación de los archivos se define en un solo lugar. La clase también crea la carpeta data/ si no existe.
Repositorios que reciben un Path en el constructor	Permite usar otro archivo, por ejemplo en pruebas, sin modificar el código.
Opción 21 y modo --demo	Permite reproducir las evidencias con los mismos datos. Reinicia los datos y ejecuta los 20 requerimientos en orden.

3. Diagrama de clases UML

El diagrama de clases representa las 13 clases del sistema con sus atributos, métodos, visibilidades y relaciones. El signo - indica visibilidad privada y el signo + visibilidad pública; los miembros estáticos se muestran subrayados.

Por su tamaño, el diagrama se presenta en una vista general por paquetes (Figura 1) y en el detalle de cada capa (Figuras 2 a 5). El diagrama completo se incluye en el Anexo A.

Para una revisión con mayor claridad, las imágenes en alta resolución se encuentran en el repositorio del proyecto, en la carpeta docs/1. diagramas-uml/, con el mismo número de figura que tienen en este informe:

https://github.com/Brayanzamora/Sistema_gestion_valorizacion_inventario/tree/main/docs/1.diagramas-uml

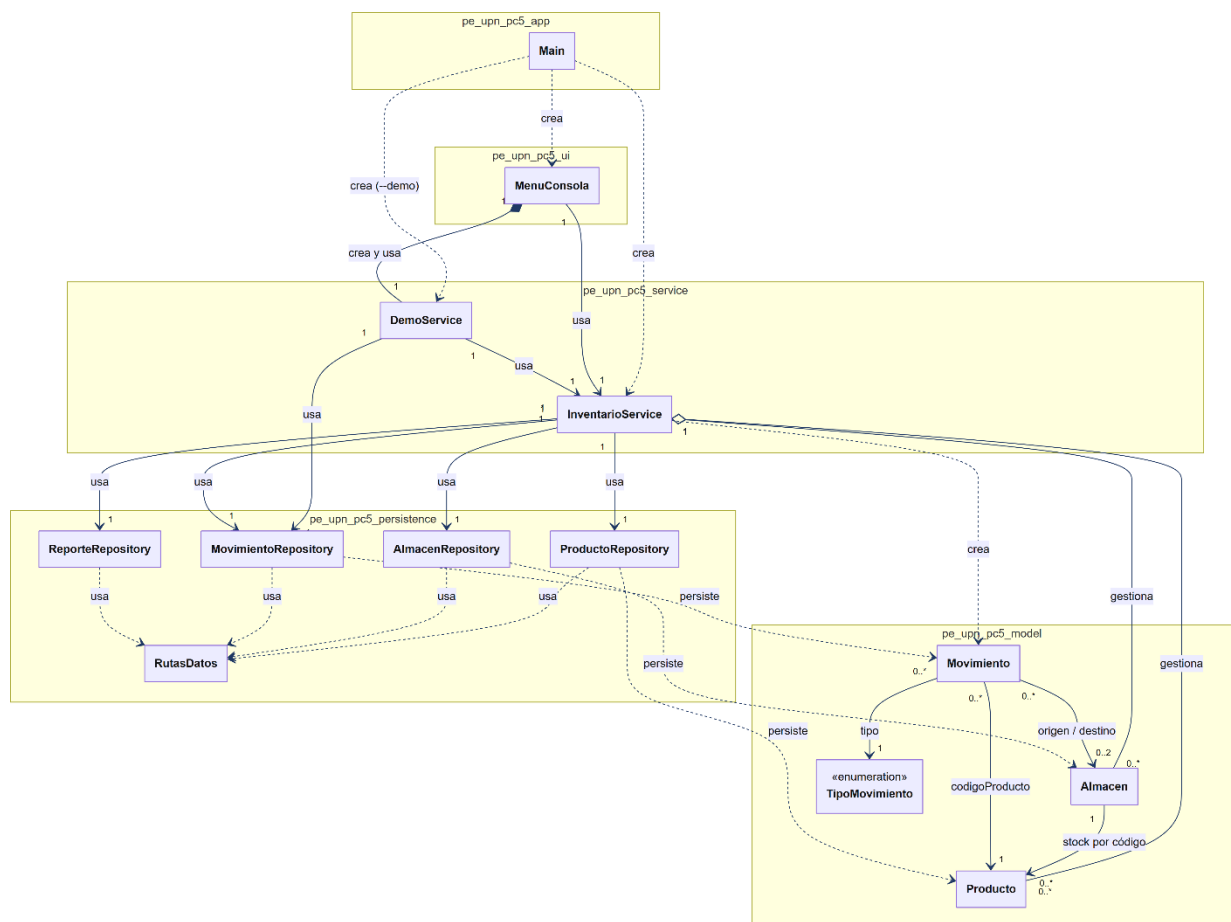


Figura 1. Vista general del diagrama de clases, con las clases agrupadas por paquete y sus relaciones.

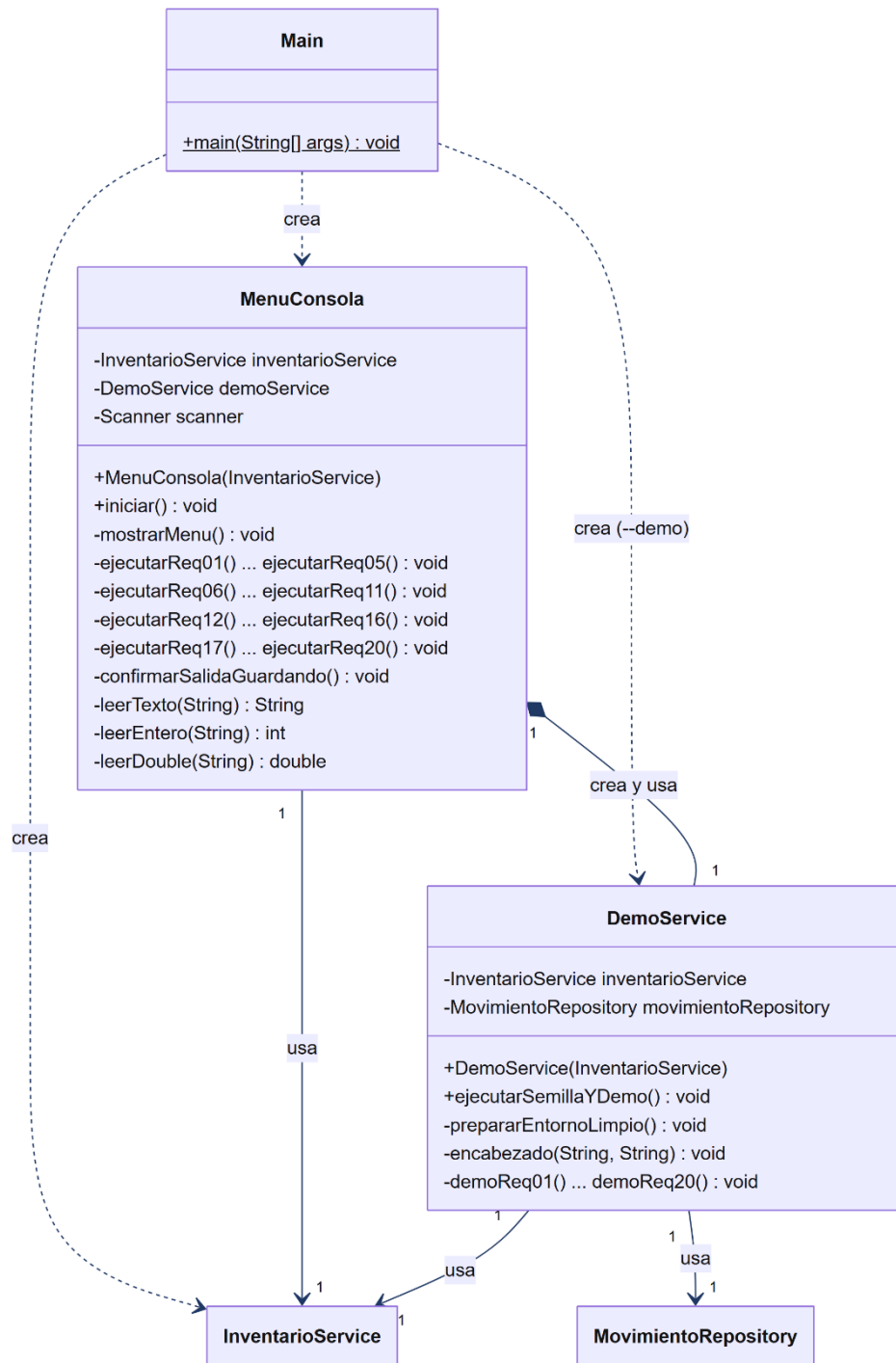


Figura 2. Capa de aplicación e interfaz: Main, MenuConsola y DemoService. Los métodos ejecutarReq01 a ejecutarReq20 y demoReq01 a demoReq20 se muestran agrupados.

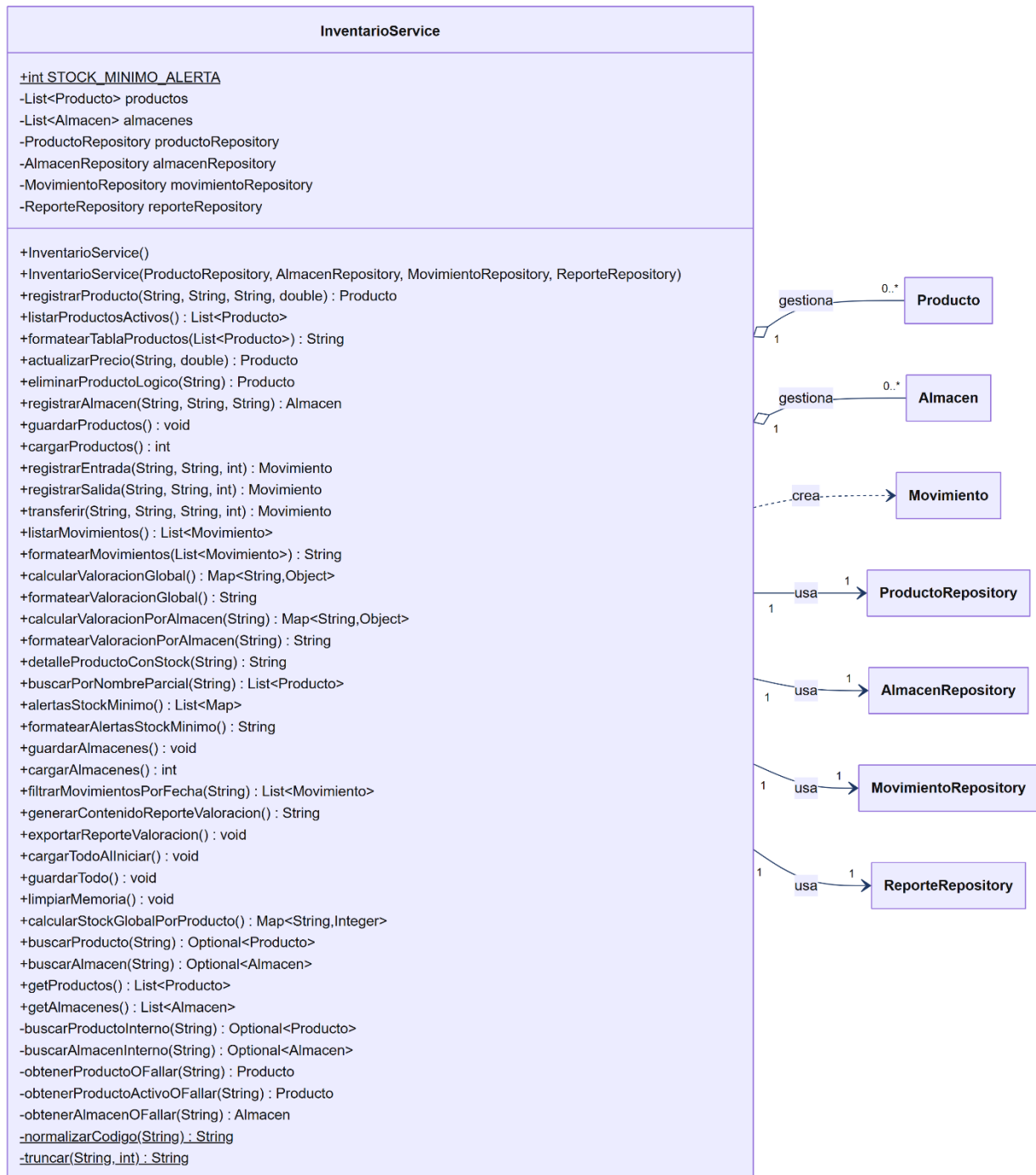


Figura 3. Capa de servicio: InventarioService.

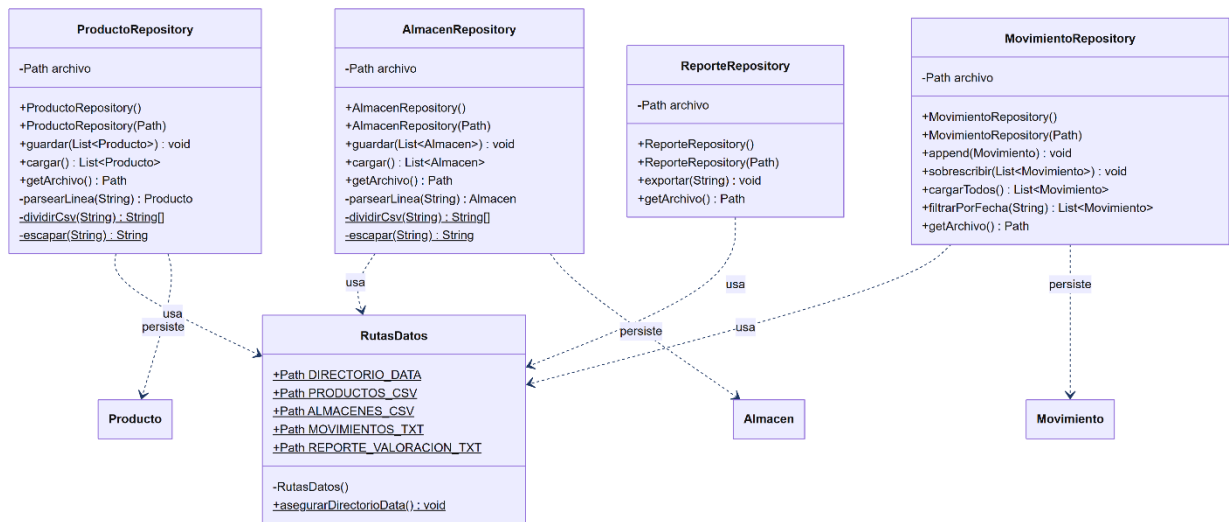


Figura 4. Capa de persistencia: repositorios y RutasDatos.

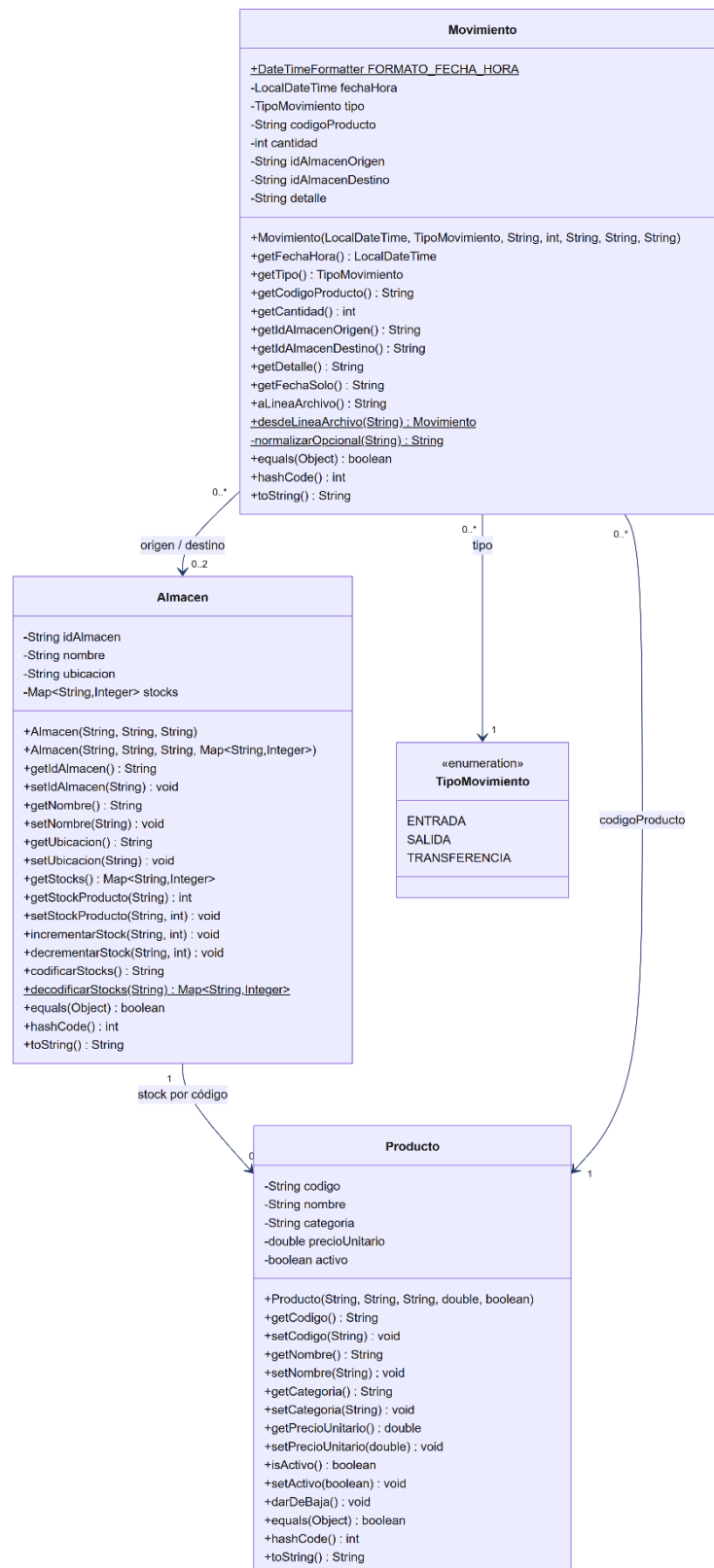


Figura 5. Capa de dominio: Producto, Almacen, Movimiento y TipoMovimiento.

3.1 Relaciones y multiplicidades

Clases relacionadas	Tipo de relación	Multiplicidad	Qué significa
Main con MenuConsola, InventarioService y DemoService	Dependencia (crea)	1 a 1	Main crea los objetos principales al arrancar. DemoService solo se crea cuando se ejecuta con --demo.
MenuConsola con InventarioService	Asociación (usa)	1 a 1	El menú le pasa al servicio todas las operaciones.
MenuConsola con DemoService	Asociación (crea y usa)	1 a 1	Se usa en la opción 21 del menú.
InventarioService con Producto	Agregación	1 a 0..*	El servicio guarda en memoria la lista de productos.
InventarioService con Almacen	Agregación	1 a 0..*	El servicio guarda en memoria la lista de almacenes.
InventarioService con los repositorios	Asociación (usa)	1 a 1	Hay un repositorio por cada tipo de archivo.
Almacen con Producto	Asociación (stock por código)	1 a 0..*	Cada almacén registra cuántas unidades tiene de cada producto.
Movimiento con TipoMovimiento	Asociación	0..* a 1	Todo movimiento tiene un único tipo.
Movimiento con Almacen	Asociación (origen y destino)	0..* a 0..2	Una entrada o una salida usan un almacén; una transferencia usa dos.
Movimiento con Producto	Asociación (codigoProducto)	0..* a 1	Cada movimiento corresponde a un producto.
Repositorios con RutasDatos	Dependencia	No aplica	Toman de ahí la ruta de su archivo y la carpeta data/.

4. Responsabilidades y encapsulamiento

4.1 Responsabilidades por clase

Clase	Responsabilidad principal	Atributos (visibilidad)
Producto	Representar un artículo del catálogo y validar sus datos (código, nombre y categoría no vacíos; precio > 0). Aplicar la baja lógica.	- codigo, - nombre, - categoria, - precioUnitario, - activo
Almacen	Representar una sede y controlar su stock local: incrementa, decrementa (impide stock negativo) y codifica/decodifica los stocks.	- idAlmacen, - nombre, - ubicacion, - stocks (final)
Movimiento	Registro inmutable de la bitácora; se convierte entre objeto y línea de texto.	- fechaHora, - tipo, - codigoProducto, - cantidad, - idAlmacenOrigen, - idAlmacenDestino, - detalle (todos final)
TipoMovimiento	Enumerar los tipos válidos de movimiento.	ENTRADA, SALIDA, TRANSFERENCIA
InventarioService	Aplicar las reglas de negocio de REQ-01 a REQ-20: unicidad, existencia, producto activo, cálculo de valoraciones y alertas. Orquestrar la persistencia.	+ STOCK_MINIMO_ALERTA; - productos, - almacenes, - 4 repositorios
DemoService	Cargar datos semilla y ejecutar la demostración automática de los 20 requerimientos.	- inventarioService, - movimientoRepository
ProductoRepository	Leer y escribir productos.csv, con parseo CSV que respeta comillas.	- archivo : Path (final)
AlmacenRepository	Leer y escribir almacenes.csv, incluida la columna stocksCodificados.	- archivo : Path (final)
MovimientoRepository	Agregar movimientos (append), leer la bitácora completa y filtrar por fecha.	- archivo : Path (final)
ReporteRepository	Escribir reporte_valoracion.txt.	- archivo : Path (final)
RutasDatos	Centralizar las rutas de los archivos y crear el directorio data/.	+ constantes estáticas Path; constructor privado
MenuConsola	Presentar el menú, leer y convertir entradas, y mostrar resultados y errores.	- inventarioService, - demoService, - scanner
Main	Iniciar la aplicación.	No tiene atributos

4.2 Aplicación del encapsulamiento

Los atributos de las clases de dominio son privados y solo se acceden mediante getters y setters. Los setters validan el valor antes de asignarlo; por ejemplo, `Producto.setPrecioUnitario` rechaza precios menores o iguales a cero.

El método `Almacen.getStocks()` devuelve el mapa de stock mediante `Collections.unmodifiableMap`, por lo que solo puede consultarse desde fuera de la clase. Los saldos se modifican únicamente con `incrementarStock`, `decrementarStock` o `setStockProducto`, que impiden valores negativos.

Los atributos de `Movimiento` y la ruta de cada repositorio se declaran `final`, de modo que no cambian después de su creación. Los métodos auxiliares, como `parsearLinea`, `dividirCsv`, `escapar` y `obtenerProductoActivoOFallar`, son privados. `RutasDatos` tiene un constructor privado, ya que solo contiene constantes.

```
public void setPrecioUnitario(double precioUnitario) {
    if (precioUnitario <= 0) {
        throw new IllegalArgumentException("El precio unitario debe ser mayor que cero.");
    }
    this.precioUnitario = precioUnitario;
}

public void decrementarStock(String codigoProducto, int cantidad) {
    ...
    if (actual < cantidad) {
        throw new IllegalStateException(String.format(
            "Stock insuficiente en almacén %s para producto %s. Disponible: %d, solicitado: %d.",
            idAlmacen, codigo, actual, cantidad));
    }
    setStockProducto(codigo, actual - cantidad);
}
```

5. Manejo de archivos

5.1 Clases de Java utilizadas

La lectura y escritura de archivos se implementó con las clases de `java.nio.file` junto con los lectores y escritores con buffer de `java.io`. Todas las operaciones usan codificación UTF-8 para conservar tildes y la letra ñ.

Clase o API	Uso en el proyecto
<code>java.nio.file.Path</code>	Representa la ruta de cada archivo (<code>RutasDatos.PRODUCTOS_CSV</code> , etc.).
<code>java.nio.file.Files</code>	<code>newBufferedReader</code> , <code>newBufferedWriter</code> , <code>exists</code> , <code>size</code> y <code>createDirectories</code> .
<code>java.io.BufferedReader</code>	Lectura línea por línea de CSV y TXT (<code>readLine</code>).
<code>java.io.BufferedWriter</code>	Escritura con buffer (<code>write</code> y <code>newLine</code>).
<code>StandardOpenOption.CREATE / APPEND</code>	Añade líneas a <code>movimientos.txt</code> sin sobrescribirlo.
<code>StandardCharsets.UTF_8</code>	Codificación explícita para conservar tildes y la ñ.
<code>try-with-resources</code>	Cierra los flujos automáticamente, incluso si ocurre una excepción.

5.2 Formato de los archivos

Archivo	Formato	Ejemplo de línea	Operación
<code>data/productos.csv</code>	<code>codigo,nombre,categoria,precioUnitario,activo</code>	<code>PROD01,Laptop Lenovo ThinkPad,Electrónica,2899.90,true</code>	Sobrescribir a completa / lectura
<code>data/almacenes.csv</code>	<code>idAlmacen,nombre,ubicacion,stocksCodificados</code>	<code>ALM02,Almacén Norte,Trujillo - La Libertad,PROD03:12 PROD04:8 PROD01:4</code>	Sobrescribir a completa / lectura
<code>data/movimientos.txt</code>	<code>fechaHora tipo producto cantidad origen destino detalle</code>	<code>2026-09-25 14:47:08 TRANSFERENCIA PROD01 4 ALM01 ALM02 Transferencia multialmacén</code>	Append / lectura / filtro
<code>data/reporte_valoracion.txt</code>	Reporte de texto con tablas alineadas	<code>TOTAL GLOBAL: S/ 70935.20</code>	Exportación

Los campos que contienen comas o comillas se escriben entre comillas dobles y las comillas internas se duplican, como en el producto `"Monitor Samsung 24""`. Durante la lectura se omiten la cabecera, las líneas vacías y las líneas que comienzan con `#`.

5.3 Escritura y lectura (fragmentos del código)

```
// ProductoRepository.guardar: escritura con Files y BufferedWriter
RutasDatos.asegurarDirectorioData();
try (BufferedWriter writer = Files.newBufferedWriter(archivo, StandardCharsets.UTF_8)) {
    writer.write("codigo,nombre,categoria,precioUnitario,activo");
    writer.newLine();
    for (Producto producto : productos) {
        writer.write(String.format("%s,%s,%s,%.2f,%s", escapar(producto.getCodigo()), ...));
        writer.newLine();
    }
}

// ProductoRepository.cargar: lectura con Files y BufferedReader
if (!Files.exists(archivo)) { return productos; }
try (BufferedReader reader = Files.newBufferedReader(archivo, StandardCharsets.UTF_8)) {
    String linea;
    while ((linea = reader.readLine()) != null) { ... productos.add(parsearLinea(linea)); }
}

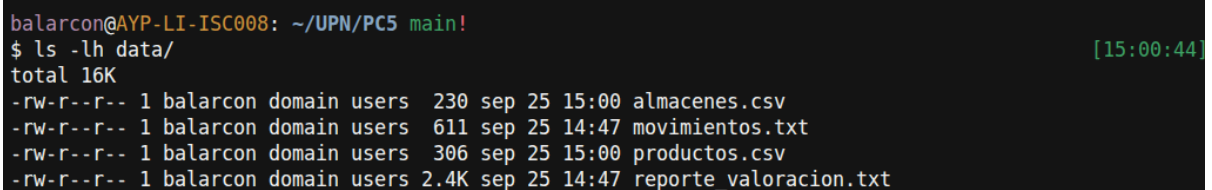
// MovimientoRepository.append: bitácora en modo APPEND
try (BufferedWriter writer = Files.newBufferedWriter(archivo, StandardCharsets.UTF_8,
    StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
    writer.write(movimiento.aLineaArchivo());
    writer.newLine();
}
```

5.4 Sincronización entre memoria y archivos

Durante la ejecución, los productos y almacenes se mantienen en memoria. Al iniciar, `cargarTodoAlIniciar()` carga `productos.csv` y `almacenes.csv`. Los movimientos, en cambio, se agregan a `movimientos.txt` en el momento en que se registran.

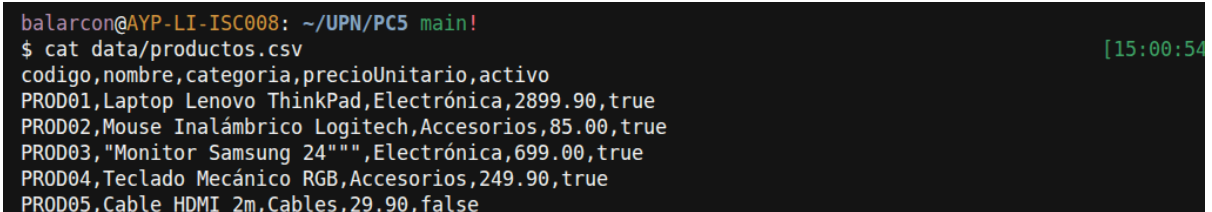
Los productos se guardan con la opción 6 y los almacenes con la opción 17. La opción 20 exporta el reporte y guarda todos los datos con `guardarTodo()`. Al salir, el menú solicita confirmación para guardar los cambios.

5.5 Evidencia de archivos generados



```
balarcon@AYP-LI-ISC008: ~/UPN/PC5 main!
$ ls -lh data/
total 16K
-rw-r--r-- 1 balarcon domain users 230 sep 25 15:00 almacenes.csv
-rw-r--r-- 1 balarcon domain users 611 sep 25 14:47 movimientos.txt
-rw-r--r-- 1 balarcon domain users 306 sep 25 15:00 productos.csv
-rw-r--r-- 1 balarcon domain users 2.4K sep 25 14:47 reporte_valoracion.txt
```

Figura 6. Archivos generados en la carpeta `data/`.



```
balarcon@AYP-LI-ISC008: ~/UPN/PC5 main!
$ cat data/productos.csv
codigo,nombre,categoria,precioUnitario,activo
PROD01,Laptop Lenovo ThinkPad,Electrónica,2899.90,true
PROD02,Mouse Inalámbrico Logitech,Accesorios,85.00,true
PROD03,"Monitor Samsung 24","",Electrónica,699.00,true
PROD04,Teclado Mecánico RGB,Accesorios,249.90,true
PROD05,Cable HDMI 2m,Cables,29.90,false
```

Figura 7. Contenido de `data/productos.csv`. Se observa el escape de comillas en `PROD03` y la baja lógica de `PROD05` (`false`).


```

data@con@ATP-LI-1SC008: ~/OPN/PCS main:
$ head -n 15 data/movimientos.txt [15:01:18]
# fechaHora | tipo | producto | cantidad | origen | destino | detalle
2026-09-25 14:47:08 | ENTRADA | PROD01 | 15 | - | ALM01 | Entrada de stock
2026-09-25 14:47:08 | ENTRADA | PROD02 | 40 | - | ALM01 | Entrada de stock
2026-09-25 14:47:08 | ENTRADA | PROD03 | 12 | - | ALM02 | Entrada de stock
2026-09-25 14:47:08 | ENTRADA | PROD04 | 8 | - | ALM02 | Entrada de stock
2026-09-25 14:47:08 | ENTRADA | PROD01 | 5 | - | ALM03 | Entrada de stock
2026-09-25 14:47:08 | SALIDA | PROD02 | 10 | ALM01 | - | Salida de stock
2026-09-25 14:47:08 | TRANSFERENCIA | PROD01 | 4 | ALM01 | ALM02 | Transferencia multialmacén

```

Figura 8. Contenido de data/movimientos.txt con cabecera y registros en formato de bitácora.

6. Matriz de trazabilidad de los 20 requerimientos

La matriz relaciona cada requerimiento con su historia de usuario, sus criterios de aceptación en formato Dado/Cuando/Entonces, los métodos Java que lo implementan, la opción del menú y la figura de evidencia. Cada opción del menú se identifica con su código [REQ-XX] (Figura 9).

ID	Historia de usuario	Criterios de aceptación	Clase / método Java	Menú	Evidencia
REQ-01 Registrar nuevo producto	Como operador de inventario, quiero registrar un nuevo producto con código único y precio válido, para mantener el catálogo actualizado.	Dado un código inexistente y un precio > 0, cuando registro el producto, entonces se agrega en memoria. Dado un código duplicado o un precio ≤ 0, cuando intento registrarlo, entonces se rechaza con un mensaje claro.	InventarioService.registrarProducto Producto.setPrecioUnitario MenuConsola.ejecutarReq01	1	Fig. 10
REQ-02 Listar productos activos	Como operador, quiero listar los productos activos en una tabla alineada, para visualizar el catálogo vigente.	Dado productos activos e inactivos, cuando elijo listar, entonces solo aparecen los activos en columnas alineadas.	InventarioService.listarProductosActivos formatearTablaProductos MenuConsola.ejecutarReq02	2	Fig. 11
REQ-03 Actualizar precio unitario	Como analista de precios, quiero actualizar el precio unitario buscando por código, para reflejar cambios de costo.	Dado un producto existente, cuando ingreso un nuevo precio > 0, entonces se actualiza. Dado un precio inválido, entonces se rechaza.	InventarioService.actualizarPrecio MenuConsola.ejecutarReq03	3	Fig. 12
REQ-04 Eliminar producto (baja lógica)	Como supervisor, quiero dar de baja lógica un producto, para ocultarlo sin perder el historial.	Dado un producto activo, cuando lo elimino lógicamente, entonces queda con activo=false y deja de listarse entre los activos, pero permanece en memoria y en el CSV.	InventarioService.eliminarProductoLogico Producto.darDeBaja MenuConsola.ejecutarReq04	4	Fig. 13

REQ-05 Registrar nuevo almacén	Como administrador de logística, quiero registrar un almacén con ID único, nombre y ubicación, para operar el stock por sede.	Dado un ID nuevo y datos válidos, cuando registro, entonces se crea el almacén. Dado un ID duplicado, entonces se rechaza.	InventarioService.registrarAlmacenes de Almacen MenuConsola.ejecutarReq05	5	Fig. 14
REQ-06 Guardar productos en CSV	Como operador, quiero guardar la lista de productos en CSV, para persistir el catálogo.	Dado productos en memoria, cuando guardo, entonces se escribe data/productos.csv con cabecera y filas.	InventarioService.guardarProductos ProductoRepository.guardar MenuConsola.ejecutarReq06	6	Fig. 15
REQ-07 Cargar productos desde CSV	Como operador, quiero cargar los productos desde CSV al iniciar y bajo demanda, para recuperar el estado guardado.	Dado un productos.csv válido, cuando cargo, entonces la memoria refleja el contenido del archivo.	InventarioService.cargarProductos ProductoRepository.cargar cargarTodoAlIniciar MenuConsola.ejecutarReq07	7	Fig. 16
REQ-08 Registrar entrada de stock	Como almacenista, quiero registrar la entrada de stock de un producto en un almacén, para incrementar el saldo local.	Dado un producto activo y un almacén existentes, cuando registro una entrada con cantidad > 0, entonces aumenta el stock y se escribe un movimiento ENTRADA.	InventarioService.registrarEntrada Almacen.incrementarStock MenuConsola.ejecutarReq08	8	Fig. 17
REQ-09 Registrar salida de stock	Como almacenista, quiero registrar salidas validando el saldo, para no permitir stock negativo.	Dado un saldo suficiente, cuando registro la salida, entonces se decrementa y se registra SALIDA. Dado un saldo insuficiente, entonces se lanza un error y el stock no se modifica.	InventarioService.registrarSalida Almacen.decrementarStock MenuConsola.ejecutarReq09	9	Fig. 18

REQ-10 Transferencia multialmacén	Como coordinador, quiero transferir stock entre almacenes en una sola operación validada, para redistribuir el inventario.	Dado origen ≠ destino y saldo suficiente, cuando transfiero, entonces se resta en el origen, se suma en el destino y se registra TRANSFERENCIA.	InventarioService.transferir MenuConsola.ejecutarReq10	10	Fig. 19
REQ-11 Bitácora de movimientos	Como auditor, quiero que cada movimiento quede en la bitácora y poder verla en consola, para tener trazabilidad.	Dado entradas, salidas y transferencias, cuando consulto el log, entonces veo en consola las líneas leídas de movimientos.txt.	MovimientoRepository.append InventarioService.listarMovimientos formatearMovimientos MenuConsola.ejecutarReq11	11	Fig. 20
REQ-12 Valoración total global	Como gerente financiero, quiero la valoración total global (stock × precio), para conocer el valor del inventario.	Dado productos activos con stock, cuando calculo la valoración global, entonces se muestra el detalle por producto y el total.	InventarioService.calcularValoracionGlobal formatearValoracionGlobal MenuConsola.ejecutarReq12	12	Fig. 21
REQ-13 Valoración por almacén	Como jefe de almacén, quiero la valoración filtrada por almacén, para evaluar esa sede.	Dado un ID de almacén válido, cuando consulto, entonces veo solo el stock y el valor de ese almacén, con su total.	InventarioService.calcularValoracionPorAlmacen formatearValoracionPorAlmacen MenuConsola.ejecutarReq13	13	Fig. 22
REQ-14 Buscar producto por código exacto	Como operador, quiero buscar un producto por código exacto con el stock desglosado, para ubicar existencias.	Dado un código existente, cuando busco, entonces veo el detalle del producto, el stock en cada almacén y el total global.	InventarioService.detalleProductoConStock MenuConsola.ejecutarReq14	14	Fig. 23
REQ-15 Buscar por	Como operador, quiero	Dado productos con nombres distintos, cuando busco un	InventarioService.buscarPorNombreParcial MenuConsola.ejecutarReq15	15	Fig. 24

nombre parcial	buscar por coincidencia parcial del nombre (sin distinguir mayúsculas), para hallar productos similares.	fragmento, entonces se listan las coincidencias sin distinguir mayúsculas y minúsculas. Un texto vacío se rechaza.			
REQ-16 Alerta de stock mínimo	Como supervisor, quiero alertas de stock mínimo (< 10 unidades globales), para reponer a tiempo.	Dado productos activos, cuando genero la alerta, entonces se listan aquellos cuyo stock sumado en todos los almacenes es < 10.	InventarioService.alertasStockMinimo formatearAlertasStockMinimo MenuConsola.ejecutarReq16	16	Fig. 25
REQ-17 Guardar almacenes en CSV	Como operador, quiero guardar los almacenes y sus saldos locales en CSV, para persistir el stock por sede.	Dado almacenes en memoria, cuando guardo, entonces se escribe data/almacenes.csv con la columna stocksCodificados.	InventarioService.guardarAlmacenes AlmacenRepository.guardarAlmacen.codificarStocks MenuConsola.ejecutarReq17	17	Fig. 26
REQ-18 Cargar almacenes desde CSV	Como operador, quiero cargar los almacenes y sus saldos desde CSV, para restaurar el inventario local.	Dado un almacenes.csv válido, cuando cargo, entonces se reconstruyen los almacenes y su mapa de stocks.	InventarioService.cargarAlmacenes AlmacenRepository.cargarAlmacen.decodificarStocks MenuConsola.ejecutarReq18	18	Fig. 27
REQ-19 Filtrar movimientos por fecha	Como auditor, quiero filtrar los movimientos por fecha YYYY-MM-DD, para revisar la operación de un día.	Dado una bitácora con varias fechas, cuando filtro por una fecha válida, entonces solo se muestran los movimientos de ese día. Un formato inválido produce un error controlado.	InventarioService.filtrarMovimientosPorFecha MovimientoRepository.filtrarPorFecha MenuConsola.ejecutarReq19	19	Fig. 28
REQ-20 Exportar reporte de valoración	Como gerente, quiero exportar el reporte	Dado un inventario valorado, cuando exporto, entonces se genera data/reporte_valoracion.txt con la valoración	InventarioService.exportarReporteValoracion generarContenidoReporteValoracion ReporteRepository.exportar MenuConsola.ejecutarReq20	20	Fig. 29

	completo de valoración a TXT, para entregar evidencia financiera.	global, la valoración por almacén y las alertas.			
--	---	--	--	--	--

Adicionalmente, la opción 21 (o el argumento --demo) ejecuta `DemoService.ejecutarSemillaYDemo`, que carga datos de prueba, ejecuta los requerimientos REQ-01 a REQ-20 en secuencia con el encabezado `=== EVIDENCIA REQ-XX ===` y guarda todos los archivos en `data/`.

7. Evidencias de funcionamiento

Las capturas se obtuvieron al ejecutar `java -cp out pe.upn.pc5.app.Main` con la opción 21. Todas parten del mismo conjunto de datos, por lo que los valores son consistentes entre requerimientos.

7.1 Menú principal

```
=====
SISTEMA DE GESTIÓN Y VALORACIÓN DE INVENTARIO MULTIALMACÉN
Práctica de Campo 5 – Java 17 | P00 + Archivos CSV/TXT
=====
Datos cargados: 5 productos, 3 almacenes.

===== MENÚ PRINCIPAL =====
1. [REQ-01] Registrar nuevo producto
2. [REQ-02] Listar productos activos
3. [REQ-03] Actualizar precio unitario
4. [REQ-04] Eliminar producto (baja lógica)
5. [REQ-05] Registrar nuevo almacén
6. [REQ-06] Guardar productos en productos.csv
7. [REQ-07] Cargar productos desde productos.csv
8. [REQ-08] Registrar entrada de stock
9. [REQ-09] Registrar salida de stock
10. [REQ-10] Transferencia multialmacén
11. [REQ-11] Ver bitácora de movimientos
12. [REQ-12] Valoración total del inventario global
13. [REQ-13] Valoración filtrada por almacén
14. [REQ-14] Buscar producto por código exacto
15. [REQ-15] Buscar productos por nombre parcial
16. [REQ-16] Alerta de stock mínimo (< 10)
17. [REQ-17] Guardar almacenes en almacenes.csv
18. [REQ-18] Cargar almacenes desde almacenes.csv
19. [REQ-19] Filtrar movimientos por fecha
20. [REQ-20] Exportar reporte_valoracion.txt
21. Ejecutar Semilla de Datos y Demo Automática
0. Salir
=====
Seleccione una opción: 
```

Figura 9. Menú principal con los 20 requerimientos identificados y la opción de demostración automática.

7.2 REQ-01: Registrar nuevo producto

Se registraron los productos PROD01 a PROD05. El sistema rechazó el registro duplicado de PROD01 y el de un producto con precio no válido.

```
=== EVIDENCIA REQ-01: REGISTRAR NUEVO PRODUCTO ===
Productos registrados:
- PROD01 | Laptop Lenovo ThinkPad | Electrónica | S/ 2899.90 | ACTIVO
- PROD02 | Mouse Inalámbrico Logitech | Accesorios | S/ 79.50 | ACTIVO
- PROD03 | Monitor Samsung 24" | Electrónica | S/ 699.00 | ACTIVO
- PROD04 | Teclado Mecánico RGB | Accesorios | S/ 249.90 | ACTIVO
- PROD05 | Cable HDMI 2m | Cables | S/ 29.90 | ACTIVO
Validación código único OK -> Ya existe un producto con código: PROD01
Validación precio > 0 OK -> El precio unitario debe ser mayor que cero.
```

Figura 10. Evidencia REQ-01: Registrar nuevo producto.

7.3 REQ-02: Listar productos activos

El listado muestra los 5 productos activos en columnas alineadas.

```
=== EVIDENCIA REQ-02: LISTAR PRODUCTOS ACTIVOS ===
```

CODIGO	NOMBRE	CATEGORIA	PRECIO	ESTADO
PROD01	Laptop Lenovo ThinkPad	Electrónica	2899.90	ACTIVO
PROD02	Mouse Inalámbrico Logitech	Accesorios	79.50	ACTIVO
PROD03	Monitor Samsung 24"	Electrónica	699.00	ACTIVO
PROD04	Teclado Mecánico RGB	Accesorios	249.90	ACTIVO
PROD05	Cable HDMI 2m	Cables	29.90	ACTIVO

Figura 11. Evidencia REQ-02: Listar productos activos.

7.4 REQ-03: Actualizar precio unitario

El precio de PROD02 pasó de S/ 79.50 a S/ 85.00.

```
=== EVIDENCIA REQ-03: ACTUALIZAR PRECIO UNITARIO ===
```

Precio actualizado de PROD02: S/ 85.00

CODIGO	NOMBRE	CATEGORIA	PRECIO	ESTADO
PROD02	Mouse Inalámbrico Logitech	Accesorios	85.00	ACTIVO

Figura 12. Evidencia REQ-03: Actualizar precio unitario.

7.5 REQ-04: Eliminar producto (baja lógica)

PROD05 deja de figurar entre los productos activos y se conserva en el listado completo con estado INACTIVO.

```
=== EVIDENCIA REQ-04: ELIMINAR PRODUCTO (BAJA LÓGICA) ===
```

Producto dado de baja lógica: PROD05 -> activo=false

Listado de activos (PROD05 no aparece):

CODIGO	NOMBRE	CATEGORIA	PRECIO	ESTADO
PROD01	Laptop Lenovo ThinkPad	Electrónica	2899.90	ACTIVO
PROD02	Mouse Inalámbrico Logitech	Accesorios	85.00	ACTIVO
PROD03	Monitor Samsung 24"	Electrónica	699.00	ACTIVO
PROD04	Teclado Mecánico RGB	Accesorios	249.90	ACTIVO

Historial completo aún conserva PROD05:

CODIGO	NOMBRE	CATEGORIA	PRECIO	ESTADO
PROD01	Laptop Lenovo ThinkPad	Electrónica	2899.90	ACTIVO
PROD02	Mouse Inalámbrico Logitech	Accesorios	85.00	ACTIVO
PROD03	Monitor Samsung 24"	Electrónica	699.00	ACTIVO
PROD04	Teclado Mecánico RGB	Accesorios	249.90	ACTIVO
PROD05	Cable HDMI 2m	Cables	29.90	INACTIVO

Figura 13. Evidencia REQ-04: Eliminar producto (baja lógica).

7.6 REQ-05: Registrar nuevo almacén

Se registraron los almacenes ALM01, ALM02 y ALM03. El registro duplicado de ALM01 fue rechazado.


```

=== EVIDENCIA REQ-05: REGISTRAR NUEVO ALMACÉN ===
Almacenes registrados:
- ALM01 | Almacén Central Lima | Lima - Ate | stocks=[]
- ALM02 | Almacén Norte | Trujillo - La Libertad | stocks=[]
- ALM03 | Almacén Sur | Arequipa - Cerro Colorado | stocks=[]
Validación ID único OK -> Ya existe un almacén con ID: ALM01

```

Figura 14. Evidencia REQ-05: Registrar nuevo almacén.

7.7 REQ-06: Guardar productos en CSV

El archivo data/productos.csv se generó con 306 bytes.

```

=== EVIDENCIA REQ-06: GUARDAR PRODUCTOS EN productos.csv ===
Archivo guardado: /home/balarcon/UPN/PC5/data/productos.csv
Bytes: 306

```

Figura 15. Evidencia REQ-06: Guardar productos en CSV.

7.8 REQ-07: Cargar productos desde CSV

Se recuperaron los 5 productos del archivo, incluido PROD05 con estado inactivo.

```

=== EVIDENCIA REQ-07: CARGAR PRODUCTOS DESDE productos.csv ===
Productos cargados en memoria: 5

```

CODIGO	NOMBRE	CATEGORIA	PRECIO	ESTADO
PROD01	Laptop Lenovo ThinkPad	Electrónica	2899.90	ACTIVO
PROD02	Mouse Inalámbrico Logitech	Accesorios	85.00	ACTIVO
PROD03	Monitor Samsung 24"	Electrónica	699.00	ACTIVO
PROD04	Teclado Mecánico RGB	Accesorios	249.90	ACTIVO
PROD05	Cable HDMI 2m	Cables	29.90	INACTIVO

Figura 16. Evidencia REQ-07: Cargar productos desde CSV.

7.9 REQ-08: Registrar entrada de stock

Se registraron 5 entradas de stock; cada una generó una línea ENTRADA en la bitácora.

```

=== EVIDENCIA REQ-08: REGISTRAR ENTRADA DE STOCK ===
Entradas registradas:
- 2026-09-25 14:47:08 | ENTRADA | PROD01 | 15 | - | ALM01 | Entrada de stock
- 2026-09-25 14:47:08 | ENTRADA | PROD02 | 40 | - | ALM01 | Entrada de stock
- 2026-09-25 14:47:08 | ENTRADA | PROD03 | 12 | - | ALM02 | Entrada de stock
- 2026-09-25 14:47:08 | ENTRADA | PROD04 | 8 | - | ALM02 | Entrada de stock
- 2026-09-25 14:47:08 | ENTRADA | PROD01 | 5 | - | ALM03 | Entrada de stock

```

Figura 17. Evidencia REQ-08: Registrar entrada de stock.

7.10 REQ-09: Registrar salida de stock

Se registró la salida de 10 unidades de PROD02. La salida de 9999 unidades fue rechazada por stock insuficiente (disponible: 30).

```

=== EVIDENCIA REQ-09: REGISTRAR SALIDA DE STOCK ===
Salida OK: 2026-09-25 14:47:08 | SALIDA | PROD02 | 10 | ALM01 | - | Salida de stock
Validación stock insuficiente OK -> Stock insuficiente en almacén ALM01 para producto PROD02. Disponibl
e: 30, solicitado: 9999.

```

Figura 18. Evidencia REQ-09: Registrar salida de stock.

7.11 REQ-10: Transferencia multialmacén

Se transfirieron 4 unidades de PROD01 de ALM01 a ALM02. Saldos resultantes: ALM01 con 11 y ALM02 con 4.

```

=== EVIDENCIA REQ-10: TRANSFERENCIA MULTIALMACÉN ===
Transferencia OK: 2026-09-25 14:47:08 | TRANSFERENCIA | PROD01 | 4 | ALM01 | ALM02 | Transferencia mult
ialmacén
Stock PROD01 en ALM01: 11
Stock PROD01 en ALM02: 4

```

Figura 19. Evidencia REQ-10: Transferencia multialmacén.

7.12 REQ-11: Bitácora de movimientos

La consola muestra los 7 movimientos leídos desde data/movimientos.txt.

```

=== EVIDENCIA REQ-11: BITÁCORA DE MOVIMIENTOS ===
FECHA/HORA      TIPO      PRODUCTO      CANT ORIGEN      DESTINO      DETALLE
-----
2026-09-25 14:47:08 ENTRADA      PROD01          15 -            ALM01      Entrada de stock
2026-09-25 14:47:08 ENTRADA      PROD02          40 -            ALM01      Entrada de stock
2026-09-25 14:47:08 ENTRADA      PROD03          12 -            ALM02      Entrada de stock
2026-09-25 14:47:08 ENTRADA      PROD04           8 -            ALM02      Entrada de stock
2026-09-25 14:47:08 ENTRADA      PROD01           5 -            ALM03      Entrada de stock
2026-09-25 14:47:08 SALIDA      PROD02          10 ALM01          -          Salida de stock
2026-09-25 14:47:08 TRANSFERENCIA PROD01           4 ALM01          ALM02      Transferencia multialma...
Archivo: /home/balarcon/UPN/PC5/data/movimientos.txt

```

Figura 20. Evidencia REQ-11: Bitácora de movimientos.

7.13 REQ-12: Valoración total global

La valoración total del inventario es S/ 70,935.20.

```

=== EVIDENCIA REQ-12: VALORACIÓN TOTAL GLOBAL ===
VALORACIÓN TOTAL DEL INVENTARIO GLOBAL
CODIGO      NOMBRE      STOCK      PRECIO      VALOR
-----
PROD01      Laptop Lenovo ThinkPad      20      2899.90      57998.00
PROD02      Mouse Inalámbrico Logitech      30      85.00      2550.00
PROD03      Monitor Samsung 24"      12      699.00      8388.00
PROD04      Teclado Mecánico RGB      8      249.90      1999.20
-----
TOTAL GLOBAL: S/ 70935.20

```

Figura 21. Evidencia REQ-12: Valoración total global.

7.14 REQ-13: Valoración por almacén

La valoración de ALM01 es S/ 34,448.90 y la de ALM02, S/ 21,986.80.

```

=== EVIDENCIA REQ-13: VALORACIÓN POR ALMACÉN ===
VALORACIÓN DEL ALMACÉN ALM01 - Almacén Central Lima (Lima - Ate)

```

CODIGO	NOMBRE	STOCK	PRECIO	VALOR
PROD01	Laptop Lenovo ThinkPad	11	2899.90	31898.90
PROD02	Mouse Inalámbrico Logitech	30	85.00	2550.00

```

TOTAL ALMACÉN: S/ 34448.90

VALORACIÓN DEL ALMACÉN ALM02 - Almacén Norte (Trujillo - La Libertad)

```

CODIGO	NOMBRE	STOCK	PRECIO	VALOR
PROD01	Laptop Lenovo ThinkPad	4	2899.90	11599.60
PROD03	Monitor Samsung 24"	12	699.00	8388.00
PROD04	Teclado Mecánico RGB	8	249.90	1999.20

```

TOTAL ALMACÉN: S/ 21986.80

```

Figura 22. Evidencia REQ-13: Valoración por almacén.

7.15 REQ-14: Buscar producto por código exacto

PROD01 registra 11 unidades en ALM01, 4 en ALM02 y 5 en ALM03, con un stock global de 20.

```

=== EVIDENCIA REQ-14: BUSCAR PRODUCTO POR CÓDIGO EXACTO ===
DETALLE DE PRODUCTO

```

Código	: PROD01
Nombre	: Laptop Lenovo ThinkPad
Categoría	: Electrónica
Precio	: S/ 2899.90
Estado	: ACTIVO

```

STOCK POR ALMACÉN

```

ID	NOMBRE	STOCK
ALM01	Almacén Central Lima	11
ALM02	Almacén Norte	4
ALM03	Almacén Sur	5

```

STOCK GLOBAL: 20

```

Figura 23. Evidencia REQ-14: Buscar producto por código exacto.

7.16 REQ-15: Buscar por nombre parcial

La búsqueda de 'mouse' devuelve PROD02 y la de 'lenovo' devuelve PROD01, sin distinguir mayúsculas y minúsculas.

```

=== EVIDENCIA REQ-15: BUSCAR PRODUCTOS POR NOMBRE PARCIAL ===
Búsqueda case-insensitive de 'mouse':

```

CODIGO	NOMBRE	CATEGORIA	PRECIO	ESTADO
PROD02	Mouse Inalámbrico Logitech	Accesorios	85.00	ACTIVO

```

Búsqueda de 'lenovo':

```

CODIGO	NOMBRE	CATEGORIA	PRECIO	ESTADO
PROD01	Laptop Lenovo ThinkPad	Electrónica	2899.90	ACTIVO

Figura 24. Evidencia REQ-15: Buscar por nombre parcial.

7.17 REQ-16: Alerta de stock mínimo

PROD04 figura en la alerta, con 8 unidades en total.

```
=== EVIDENCIA REQ-16: ALERTA DE STOCK MÍNIMO ===
ALERTA DE STOCK MÍNIMO (< 10 unidades globales)
CODIGO      NOMBRE                                STOCK
-----
PROD04      Teclado Mecánico RGB                        8
```

Figura 25. Evidencia REQ-16: Alerta de stock mínimo.

7.18 REQ-17: Guardar almacenes en CSV

El archivo almacenes.csv contiene una fila por almacén, con el stock codificado (por ejemplo, PROD01:11|PROD02:30).

```
=== EVIDENCIA REQ-17: GUARDAR ALMACENES EN almacenes.csv ===
Archivo guardado: /home/balarcon/UPN/PC5/data/almacenes.csv
Contenido:
idAlmacen,nombre,ubicacion,stocksCodificados
ALM01,Almacén Central Lima,Lima - Ate,PROD01:11|PROD02:30
ALM02,Almacén Norte,Trujillo - La Libertad,PROD03:12|PROD04:8|PROD01:4
ALM03,Almacén Sur,Arequipa - Cerro Colorado,PROD01:5
```

Figura 26. Evidencia REQ-17: Guardar almacenes en CSV.

7.19 REQ-18: Cargar almacenes desde CSV

Se reconstruyeron los 3 almacenes con sus saldos.

```
=== EVIDENCIA REQ-18: CARGAR ALMACENES DESDE almacenes.csv ===
Almacenes cargados: 3
- ALM01 | Almacén Central Lima | Lima - Ate | stocks=[PROD01:11|PROD02:30]
- ALM02 | Almacén Norte | Trujillo - La Libertad | stocks=[PROD03:12|PROD04:8|PROD01:4]
- ALM03 | Almacén Sur | Arequipa - Cerro Colorado | stocks=[PROD01:5]
```

Figura 27. Evidencia REQ-18: Cargar almacenes desde CSV.

7.20 REQ-19: Filtrar movimientos por fecha

El filtro por la fecha 2026-09-25 devuelve 7 movimientos.

```
=== EVIDENCIA REQ-19: FILTRAR MOVIMIENTOS POR FECHA ===
Filtro por fecha 2026-09-25 (7 registros):
```

FECHA/HORA	TIPO	PRODUCTO	CANT	ORIGEN	DESTINO	DETALLE
2026-09-25 14:47:08	ENTRADA	PROD01	15	-	ALM01	Entrada de stock
2026-09-25 14:47:08	ENTRADA	PROD02	40	-	ALM01	Entrada de stock
2026-09-25 14:47:08	ENTRADA	PROD03	12	-	ALM02	Entrada de stock
2026-09-25 14:47:08	ENTRADA	PROD04	8	-	ALM02	Entrada de stock
2026-09-25 14:47:08	ENTRADA	PROD01	5	-	ALM03	Entrada de stock
2026-09-25 14:47:08	SALIDA	PROD02	10	ALM01	-	Salida de stock
2026-09-25 14:47:08	TRANSFERENCIA	PROD01	4	ALM01	ALM02	Transferencia multialma...

Figura 28. Evidencia REQ-19: Filtrar movimientos por fecha.

7.21 REQ-20: Exportar reporte de valoración

Se generó reporte_valoracion.txt con la valoración total, la de cada almacén y la alerta de stock mínimo.

```
=== EVIDENCIA REQ-20: EXPORTAR REPORTE DE VALORACIÓN ===
Reporte exportado a: /home/balarcon/UPN/PC5/data/reporte_valoracion.txt
--- Vista previa (primeras líneas) ---
REPORTE DE VALORACIÓN DE INVENTARIO MULTIALMACÉN
Generado: 2026-09-25 14:47:08
=====
VALORACIÓN TOTAL DEL INVENTARIO GLOBAL
CODIGO      NOMBRE                      STOCK      PRECIO      VALOR
-----
PROD01      Laptop Lenovo ThinkPad        20          2899.90     57998.00
PROD02      Mouse Inalámbrico Logitech    30           85.00     2550.00
PROD03      Monitor Samsung 24"           12          699.00     8388.00
PROD04      Teclado Mecánico RGB          8           249.90     1999.20
-----
TOTAL GLOBAL: S/ 70935.20

VALORACIÓN DEL ALMACÉN ALM01 - Almacén Central Lima (Lima - Ate)
CODIGO      NOMBRE                      STOCK      PRECIO      VALOR
-----
PROD01      Laptop Lenovo ThinkPad        11          2899.90     31898.90
PROD02      Mouse Inalámbrico Logitech    30           85.00     2550.00
-----
TOTAL ALMACÉN: S/ 34448.90

VALORACIÓN DEL ALMACÉN ALM02 - Almacén Norte (Trujillo - La Libertad)
CODIGO      NOMBRE                      STOCK      PRECIO      VALOR
-----
PROD01      Laptop Lenovo ThinkPad         4          2899.90     11599.60
... (17 líneas más)

=== DEMO COMPLETADA ===
Archivos generados en carpeta data/:
- /home/balarcon/UPN/PC5/data/productos.csv
- /home/balarcon/UPN/PC5/data/almacenes.csv
- /home/balarcon/UPN/PC5/data/movimientos.txt
- /home/balarcon/UPN/PC5/data/reporte_valoracion.txt

Presione ENTER para continuar...^C
```

Figura 29. Evidencia REQ-20: Exportar reporte de valoración.

8. Registro de pruebas

Las pruebas se realizaron después de cada incremento e incluyen casos válidos y casos de error. La evidencia de cada caso se encuentra en las secciones 7 y 9.

ID	REQ	Caso de prueba	Resultado esperado	Resultado obtenido
P01	REQ-01	Registrar PROD01 a PROD05 con datos válidos	Se registran 5 productos	Conforme
P02	REQ-01	Registrar de nuevo PROD01	Rechazo: "Ya existe un producto con código: PROD01"	Conforme
P03	REQ-01	Registrar un producto con precio ≤ 0	Rechazo: "El precio unitario debe ser mayor que cero."	Conforme
P04	REQ-01	Ingresar el precio "abc" desde el menú	Error de formato numérico controlado; el programa continúa	Conforme
P05	REQ-02	Listar activos	5 productos en tabla alineada	Conforme
P06	REQ-03	Cambiar el precio de PROD02 a 85.00	El precio se actualiza	Conforme
P07	REQ-04	Baja lógica de PROD05	No aparece entre los activos; sigue en el historial como INACTIVO	Conforme
P08	REQ-05	Registrar ALM01 a ALM03 y repetir ALM01	3 almacenes registrados; el duplicado se rechaza	Conforme
P09	REQ-06	Guardar productos	productos.csv generado (306 bytes)	Conforme
P10	REQ-07	Cargar productos desde el CSV	5 productos en memoria, incluido el inactivo	Conforme
P11	REQ-08	5 entradas de stock	Stock incrementado y 5 líneas ENTRADA en la bitácora	Conforme
P12	REQ-09	Salida de 10 u. de PROD02 en ALM01	Stock de 40 a 30; línea SALIDA	Conforme
P13	REQ-09	Salida de 9999 u. de PROD02 (demo) y de 999999 u. de PROD01 (menú)	Rechazo por stock insuficiente; el stock no cambia	Conforme
P14	REQ-10	Transferir 4 u. de PROD01 de ALM01 a ALM02	ALM01 = 11, ALM02 = 4; línea TRANSFERENCIA	Conforme

P15	REQ-11	Consultar la bitácora	7 movimientos leídos de movimientos.txt	Conforme
P16	REQ-12	Valoración global	S/ 70,935.20	Conforme
P17	REQ-13	Valoración de ALM01 y ALM02	S/ 34,448.90 y S/ 21,986.80	Conforme
P18	REQ-14	Buscar PROD01	Stock por almacén 11 / 4 / 5; global 20	Conforme
P19	REQ-15	Buscar 'mouse' y 'lenovo'	PROD02 y PROD01, respectivamente	Conforme
P20	REQ-16	Generar la alerta de stock mínimo	PROD04 (8 u.)	Conforme
P21	REQ-17	Guardar almacenes	almacenes.csv con los stocks codificados	Conforme
P22	REQ-18	Cargar almacenes	3 almacenes con su mapa de stock reconstruido	Conforme
P23	REQ-19	Filtrar por 2026-09-25	7 registros	Conforme
P24	REQ-20	Exportar el reporte	reporte_valoracion.txt (2.4 KB) con las 3 secciones	Conforme

9. Control de errores

Las clases del modelo y del servicio lanzan excepciones ante datos o estados inválidos, y el menú las captura en un único punto. El usuario recibe un mensaje descriptivo y el programa continúa su ejecución.

Tipo de error	Excepción	Dónde se origina	Tratamiento
Lectura/escritura de archivos	IOException	Repositorios (Files.newBuffered*)	MenuConsola: "Error de E/S de archivos: ...". Al iniciar y al salir se muestra una advertencia y la ejecución continúa.
Formato numérico en la entrada del usuario	NumberFormatException	MenuConsola.leerEntero / leerDouble	"Error de formato numérico: Se esperaba un ..."
Formato numérico en un CSV	NumberFormatException, que se relanza como IllegalArgumentException	ProductoRepository.parsearLinea, Almacen.decodificarStocks	Mensaje "Precio inválido en producto: ..." o "Formato de stock inválido"
Línea corrupta en la bitácora	IllegalArgumentException	Movimiento.desdeLineaArchivo	La línea se omite con una advertencia y se sigue leyendo el resto del archivo
Datos inválidos	IllegalArgumentException	Setters de Producto y Almacen, InventarioService	"Operación rechazada: ..." (código vacío, precio ≤ 0 , cantidad ≤ 0 , código o ID duplicado, fecha con formato inválido, texto de búsqueda vacío)
Estado inválido	IllegalStateException	Almacen.decrementarStock, InventarioService	"Operación rechazada: Stock insuficiente...", producto inactivo o baja repetida
Imprevisto	Exception	Cualquier parte	"Error inesperado: ..." sin cerrar el programa

```

} catch (NumberFormatException e) {
    System.out.println("Error de formato numérico: " + e.getMessage());
} catch (IllegalArgumentException | IllegalStateException e) {
    System.out.println("Operación rechazada: " + e.getMessage());
} catch (IOException e) {
    System.out.println("Error de E/S de archivos: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Error inesperado: " + e.getMessage());
}

```

9.1 Evidencias de errores controlados


```

===== MENÚ PRINCIPAL =====
1. [REQ-01] Registrar nuevo producto
2. [REQ-02] Listar productos activos
3. [REQ-03] Actualizar precio unitario
4. [REQ-04] Eliminar producto (baja lógica)
5. [REQ-05] Registrar nuevo almacén
6. [REQ-06] Guardar productos en productos.csv
7. [REQ-07] Cargar productos desde productos.csv
8. [REQ-08] Registrar entrada de stock
9. [REQ-09] Registrar salida de stock
10. [REQ-10] Transferencia multialmacén
11. [REQ-11] Ver bitácora de movimientos
12. [REQ-12] Valoración total del inventario global
13. [REQ-13] Valoración filtrada por almacén
14. [REQ-14] Buscar producto por código exacto
15. [REQ-15] Buscar productos por nombre parcial
16. [REQ-16] Alerta de stock mínimo (< 10)
17. [REQ-17] Guardar almacenes en almacenes.csv
18. [REQ-18] Cargar almacenes desde almacenes.csv
19. [REQ-19] Filtrar movimientos por fecha
20. [REQ-20] Exportar reporte_valoracion.txt
21. Ejecutar Semilla de Datos y Demo Automática
0. Salir
=====
Seleccione una opción: 1
--- REQ-01: Registrar nuevo producto ---
Código: PROD-ERR
Nombre: Test-error
Categoría: cat
Precio unitario (> 0): abc
Error de formato numérico: Se esperaba un número decimal y se recibió: 'abc'

Presione ENTER para continuar...

```

Figura 30. Error controlado: al ingresar el precio "abc" se informa el error de formato numérico y el sistema continúa.

```

Presione ENTER para continuar...

===== MENÚ PRINCIPAL =====
1. [REQ-01] Registrar nuevo producto
2. [REQ-02] Listar productos activos
3. [REQ-03] Actualizar precio unitario
4. [REQ-04] Eliminar producto (baja lógica)
5. [REQ-05] Registrar nuevo almacén
6. [REQ-06] Guardar productos en productos.csv
7. [REQ-07] Cargar productos desde productos.csv
8. [REQ-08] Registrar entrada de stock
9. [REQ-09] Registrar salida de stock
10. [REQ-10] Transferencia multialmacén
11. [REQ-11] Ver bitácora de movimientos
12. [REQ-12] Valoración total del inventario global
13. [REQ-13] Valoración filtrada por almacén
14. [REQ-14] Buscar producto por código exacto
15. [REQ-15] Buscar productos por nombre parcial
16. [REQ-16] Alerta de stock mínimo (< 10)
17. [REQ-17] Guardar almacenes en almacenes.csv
18. [REQ-18] Cargar almacenes desde almacenes.csv
19. [REQ-19] Filtrar movimientos por fecha
20. [REQ-20] Exportar reporte_valoracion.txt
21. Ejecutar Semilla de Datos y Demo Automática
0. Salir
=====
Seleccione una opción: 9
--- REQ-09: Salida de stock ---
Código producto: PROD01
ID almacén: ALM01
Cantidad: 999999
Operación rechazada: Stock insuficiente en almacén ALM01 para producto PROD01. Disponible: 11, solicita
do: 999999.

Presione ENTER para continuar...

```

Figura 31. Error controlado: una salida de 999999 unidades se rechaza por stock insuficiente (disponible: 11).

10. Dificultades técnicas resueltas

Campos con comas o comillas en el CSV. Nombres como Monitor Samsung 24" alteraban las columnas del archivo. Se implementaron dos métodos privados en los repositorios: `escapar()`, que encierra el campo entre comillas y duplica las comillas internas, y `dividirCsv()`, que recorre la línea carácter por carácter para distinguir las comas separadoras de las que forman parte del texto.

Stock por almacén en un archivo plano. El stock se almacena en una sola columna con el formato `PROD01:20|PROD02:15`. Al leerlo, `decodificarStocks()` valida que cada par tenga código y cantidad no negativa; si el formato es incorrecto, se lanza una excepción y no se cargan datos parciales.

Consistencia en las transferencias. El método `transferir()` valida primero que el producto esté activo, que ambos almacenes existan y sean distintos, y que la cantidad sea mayor que cero. Luego descuenta del origen; si el saldo es insuficiente, la operación falla antes de modificar el destino.

Otras situaciones resueltas:

Situación	Solución
La primera vez que se ejecuta, los archivos no existen	Si el archivo no está, la carga devuelve una lista vacía. Antes de escribir, <code>RutasDatos.asegurarDirectorioData()</code> crea la carpeta <code>data/</code> .
Cabeceras y líneas vacías en los archivos	La lectura salta la cabecera, las líneas en blanco y las que empiezan con <code>#</code> .
Una línea dañada en la bitácora	Se muestra una advertencia, se ignora esa línea y se sigue leyendo el resto.
Números escritos con coma decimal, como 29,90	<code>leerDouble</code> cambia la coma por punto antes de convertir el valor.
Tildes y la letra ñ	Todas las lecturas y escrituras usan UTF-8, y el <code>pom.xml</code> también lo declara.
Repetir las evidencias con los mismos datos	La opción 21 reinicia los datos y ejecuta siempre el mismo escenario.

11. Repositorio y trazabilidad de aportes

URL del repositorio: https://github.com/Brayanzamora/Sistema_gestion_valorizacion_inventario

El desarrollo se registró en commits por capa o módulo, en el orden de construcción: proyecto base, modelo, persistencia, servicio, menú, demostración y documentación.

Commit	Mensaje	Contenido
9cb9a47	chore: inicializar proyecto Java 17 del inventario multialmacén	pom.xml, .gitignore y README
38004f1	feat: agregar entidades de dominio Producto, Almacen y Movimiento	Paquete model
bf5e441	feat: definir rutas centralizadas de archivos CSV y TXT	RutasDatos
654384e	feat: implementar persistencia de productos, almacenes, movimientos y reportes	Paquete persistence
69bc7ae	feat: implementar servicio de inventario con validaciones REQ-01 a REQ-20	InventarioService
836e9d5	feat: agregar menú de consola interactivo y punto de entrada Main	MenuConsola, Main
72279de	test: agregar semilla de datos, demo automática y evidencias en data/	DemoService y data/
c0476b1	docs: agregar diagrama UML Mermaid e informe técnico con matriz de trazabilidad	DIAGRAMA_UML.md, INFORME_TECNICO_PC5.md

```

commit c0476b1d8c6a109055cc3c8ea058b0f82df108dd (HEAD -> main, origin/main)
Author: Brian <balarcon@aynitech.com>
Date:   Fri Sep 25 14:35:39 2026 -0500

    docs: agregar diagrama UML Mermaid e informe técnico con matriz de trazabilidad

commit 72279de6019679419362cdc08e7d96c20f93e0f9
Author: Brian <balarcon@aynitech.com>
Date:   Fri Sep 25 14:35:39 2026 -0500

    test: agregar semilla de datos, demo automática y evidencias en data/

commit 836e9d50c5269aa5f89ed0c6ea42908c75acf3da
Author: Brian <balarcon@aynitech.com>
Date:   Fri Sep 25 14:35:39 2026 -0500

    feat: agregar menú de consola interactivo y punto de entrada Main

commit 69bc7ae4570aecb927a810f8e79685999bf00035
Author: Brian <balarcon@aynitech.com>
Date:   Fri Sep 25 14:35:39 2026 -0500

    feat: implementar servicio de inventario con validaciones REQ-01 a REQ-20

commit 654384e83f6e5c9d1b6afb206d0fd5c13849b7e6
Author: Brian <balarcon@aynitech.com>
Date:   Fri Sep 25 14:35:39 2026 -0500

    feat: implementar persistencia de productos, almacenes, movimientos y reportes

commit bf5e4410a770bad292d84936f065c149c84d39c9
Author: Brian <balarcon@aynitech.com>
Date:   Fri Sep 25 14:35:39 2026 -0500

    feat: definir rutas centralizadas de archivos CSV y TXT

commit 38004f1d4509b0ff070e65b104ecefef494984f29
Author: Brian <balarcon@aynitech.com>
Date:   Fri Sep 25 14:35:39 2026 -0500

    feat: agregar entidades de dominio Producto, Almacen y Movimiento

commit 9cb9a471607a0c624b49ec1acalc5356f2110d8
Author: Brian <balarcon@aynitech.com>
Date:   Fri Sep 25 14:35:39 2026 -0500

    chore: inicializar proyecto Java 17 del inventario multialmacén

~
[END]

```

Figura 32. Historial de commits del repositorio (git log).

12. Compilación y ejecución

```
# Con Maven
mvn -q compile
mvn -q exec:java                # menú interactivo
mvn -q exec:java -Dexec.args="--demo"  # demo automática REQ-01..REQ-20

# Con javac
mkdir -p out
javac -encoding UTF-8 -d out $(find src/main/java -name "*.java")
java -cp out pe.upn.pc5.app.Main      # menú interactivo
java -cp out pe.upn.pc5.app.Main --demo  # demo automática
```

Los archivos se generan en la carpeta data/, relativa al directorio de ejecución.

13. Cumplimiento de requerimientos mínimos, entregables y rúbrica

13.1 Requerimientos mínimos

N.º	Requerimiento mínimo	Cumplimiento	Sección
1	Diagrama de clases UML con clases, atributos, métodos, visibilidades y relaciones	13 clases con visibilidades, multiplicidades y relaciones (asociación, agregación, dependencia)	3
2	Responsabilidades y encapsulamiento	Atributos private, setters con validación, colecciones no modificables, atributos final	2, 4
3	Manejo de archivos con Files, BufferedReader y BufferedWriter	Se usan en los 4 repositorios, con try-with-resources y UTF-8	5
4	Guardar y recuperar información en TXT o CSV	2 CSV y 2 TXT, con lectura y escritura	5, 7
5	Criterios de aceptación de los 20 requerimientos	Formato Dado/Cuando/Entonces para REQ-01 a REQ-20	6
6	Implementar los 20 requerimientos y relacionarlos con las historias de usuario	Matriz de trazabilidad que une cada historia de usuario con sus criterios, su método Java y su evidencia	6, 7
7	Controlar errores de lectura, escritura, formato y datos inválidos	IOException, NumberFormatException, IllegalArgumentException e IllegalStateException	9
8	Actualizar el repositorio, el informe y la matriz	Repositorio en GitHub con 8 commits, este informe y la matriz	6, 11

13.2 Entregables

N.º	Entregable	Ubicación
1	Diagrama de clases UML legible y actualizado	Sección 3 y Anexo A. En el repositorio, carpeta docs/1.diagramas-uml/ (imágenes PNG)
2	Código Java con las clases del dominio y manejo de archivos funcional	Repositorio (src/main/java/pe/upn/pc5)
3	Matriz de 20 requerimientos con historias de usuario y criterios de aceptación	Sección 6
4	Evidencias de pruebas de lectura, escritura, validación y funcionamiento	Secciones 5.5, 7, 8 y 9.1
5	Informe técnico con decisiones de diseño y dificultades resueltas	Secciones 2 y 10

6	Repositorio actualizado con trazabilidad de aportes individuales	Sección 11
---	--	------------

13.3 Relación con los criterios de la rúbrica

Criterio de la rúbrica	Peso	Evidencia en este informe
1. Diagrama y diseño de clases	4	Sección 3 (diagrama y multiplicidades) y sección 4 (responsabilidades)
2. Manejo de archivos	3	Sección 5 (API, formatos, código, evidencias) y control de E/S en la sección 9
3. Criterios de aceptación	3	Sección 6 (Dado/Cuando/Entonces por requerimiento)
4. Implementación de 20 requerimientos	5	Secciones 7 y 8 (20 evidencias y 24 pruebas)
5. POO, validación y calidad del código	2	Secciones 2, 4 y 9
6. Documentación y trazabilidad	3	Secciones 6, 11 y 13

14. Conclusiones

Se obtuvo una primera versión ejecutable del sistema que cumple los 20 requerimientos priorizados, cada uno con su historia de usuario, criterios de aceptación y evidencia de funcionamiento.

La separación en capas facilitó el mantenimiento y la trazabilidad, ya que cada requerimiento se implementa en un método concreto de la capa de servicio.

El uso de Files, BufferedReader y BufferedWriter permitió guardar y recuperar el estado completo del sistema, incluido el stock por almacén y el historial de movimientos.

Las validaciones en las clases del dominio impiden estados inválidos, como precios no positivos o stock negativo, y el manejo centralizado de excepciones evita que el programa se detenga ante datos incorrectos.

Recomendaciones para la siguiente iteración

Para la siguiente versión se recomienda agregar pruebas unitarias con JUnit para las reglas de InventarioService, migrar la persistencia a una base de datos manteniendo la interfaz de los repositorios e incorporar una interfaz gráfica, en línea con la Unidad III del curso.

Diagrama completo con todas las clases, atributos, métodos y relaciones. Para revisarlo con mayor claridad, la imagen en alta resolución se encuentra en el repositorio como docs/1.diagramas-uml/Anexo_A_UML_completo.png.

